Estimating How Turbulence Spreads Out Cloud Droplet Sizes

A steady-state example: the equation, the numerical scheme, the loss, the adjoint gradient, and the optimizer

September 2026

Contents

1 What we are trying to do **2**

2 The equation **2**
- 2.1 Describing many droplets at once . . . 2
- 2.2 How a single droplet grows . . . 3
- 2.3 Turbulence makes growth random . . . 3
- 2.4 Falling out . . . 4
- 2.5 Putting it together . . . 4

3 Solving the equation on a computer **5**
- 3.1 Cells . . . 5
- 3.2 A balance for every cell . . . 6
- 3.3 The flux between two cells . . . 6
- 3.4 The linear system . . . 6
- 3.5 How fine the grid must be . . . 7

4 Setting up the estimation problem **7**

5 The gradient, by the adjoint method **8**
- 5.1 Differentiating the model . . . 8
- 5.2 The adjoint trick . . . 8
- 5.3 The derivative of the matrix . . . 9
- 5.4 Checking the gradient . . . 9
- 5.5 What the adjoint means . . . 9

6 Minimizing the loss **10**

7 Results: many true values, many starting guesses **11**

8 Summary **11**

A Parameter values and formulas **12**

1 What we are trying to do

In a cloud, tiny water droplets grow by absorbing water vapor from the surrounding air. If every droplet saw exactly the same conditions, droplets born at the same time would stay the same size. Turbulence breaks this symmetry: some droplets wander through moister air and grow faster, others through drier air and grow slower. The sizes spread out.

We describe that spreading with a single number, the **diffusivity** D_x. It can't be measured directly, but it leaves a clear fingerprint on the *distribution of droplet sizes*. The goal is to recover D_x from that distribution.

This document works through the simplest version of the problem:

- the chamber is in a **steady state**, so the distribution no longer changes in time;
- the data are the **complete distribution**, with **no noise**;
- D_x is a single **constant**, and it is the only unknown.

To make the answer known, we generate the data ourselves from the same model with a chosen "true" D_x, then check that the method finds it. Figure 1 shows the loop.

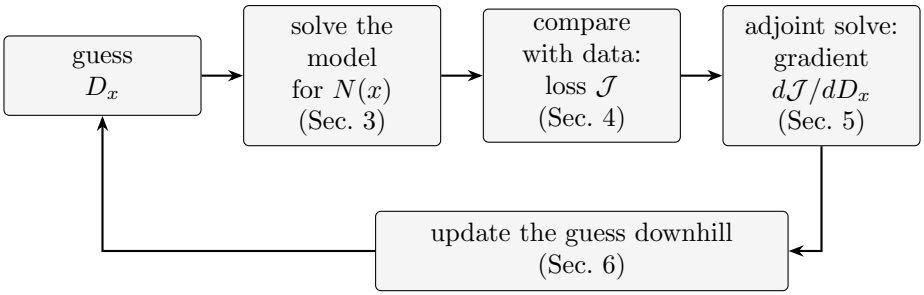


Figure 1: The estimation loop. Each pass solves the model once and the adjoint problem once, then moves the guess downhill.

2 The equation

2.1 Describing many droplets at once

A chamber contains an enormous number of droplets, so we don't track them one by one. We track how many there are of each size. It turns out to be convenient to measure size by the **radius squared**,

$$x = r^2 \qquad (\mu\mathrm{m}^2),$$

for a reason explained in the next subsection. The **size distribution** $N(x)$ is defined so that

$$N(x)\,\mathrm{d}x = \text{number of droplets per cm}^3 \text{ of air whose } r^2 \text{ lies between } x \text{ and } x + \mathrm{d}x.$$

So N has units of $\mathrm{cm}^{-3}\,\mu\mathrm{m}^{-2}$. Adding up all sizes gives the total droplet concentration, $n = \int N(x)\,\mathrm{d}x$, in droplets per cm^3. The model describes the chamber as a whole: it has no spatial coordinate, only droplet size.

2.2 How a single droplet grows

Air that holds more water vapor than it can hold at equilibrium is *supersaturated*. The supersaturation s measures by how much; here $s = 0.09\%$. A droplet in supersaturated air grows because vapor diffuses onto its surface. Physics gives the growth rate as $r\,\mathrm{d}r/\mathrm{d}t \propto s$, which is the same as

$$\frac{\mathrm{d}(r^2)}{\mathrm{d}t} = a\,s.$$

Here $a \approx 206\,\mu\mathrm{m}^2/\mathrm{s}$ depends only on temperature and pressure (Appendix A). This is why we use $x = r^2$: in that variable, condensation growth is a *constant* rate, the same for all droplets.

Very small droplets need one correction. A curved surface evaporates more easily than a flat one, while salt dissolved in the droplet makes it grow more easily. Together these define an **equilibrium supersaturation** $s_{\mathrm{eq}}(r) = A_k/r - B_k/r^3$ (Köhler theory): the value of s at which a droplet of radius r neither grows nor shrinks. The droplet responds only to the excess over it, so

$$\frac{\mathrm{d}x}{\mathrm{d}t} = A(x) = a\left(s - s_{\mathrm{eq}}(\sqrt{x})\right). \tag{1}$$

We call $A(x)$ the **drift**. Figure 2(a) shows it. For droplets bigger than a few microns, s_{eq} is negligible and $A \approx a\,s$. Near $r \approx 1\,\mu\mathrm{m}$ the drift dips sharply. There, s_{eq} comes close to s, so droplets crawl through this size range slowly.

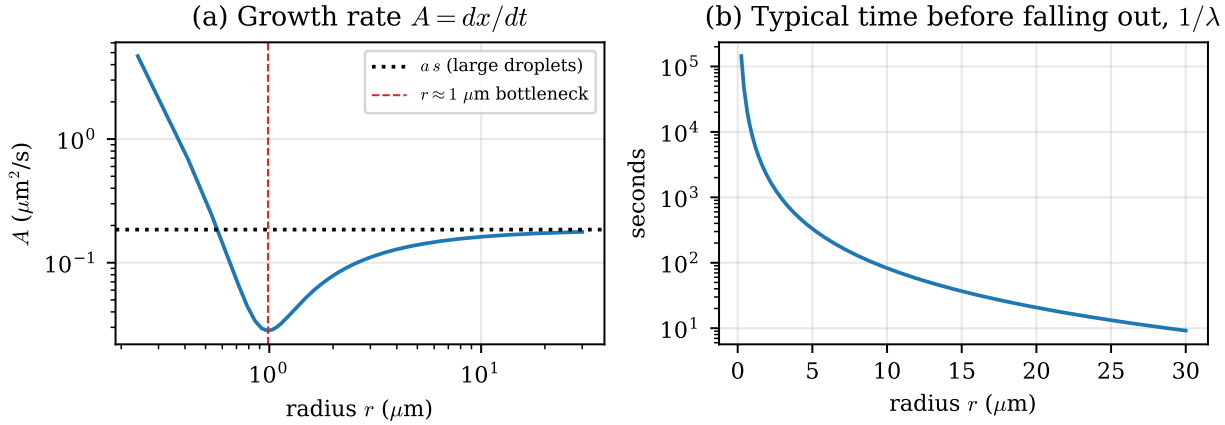


Figure 2: (a) Growth rate in $x = r^2$. Large droplets all grow at the same rate $a\,s$; droplets near $1\,\mu\mathrm{m}$ grow slowly. (b) Big droplets fall out of the chamber quickly; small ones stay for a long time.

2.3 Turbulence makes growth random

In a turbulent chamber, a droplet doesn't experience a steady s. It is carried through slightly moister and drier regions, so its growth rate fluctuates. Figure 3 simulates 25 droplets that all start at the same size. The dashed line is growth at the steady rate; the colored paths include random fluctuations. The droplets spread out over time, and they spread faster when the fluctuations are stronger.

Averaged over many droplets, this random spreading behaves exactly like **diffusion** in size space: droplets tend to move from sizes where they are crowded to sizes where they are sparse. The strength of this diffusion is D_x, in units of $\mu\mathrm{m}^4/\mathrm{s}$ (the square of the x units, per second). *This is the number we want to estimate.*

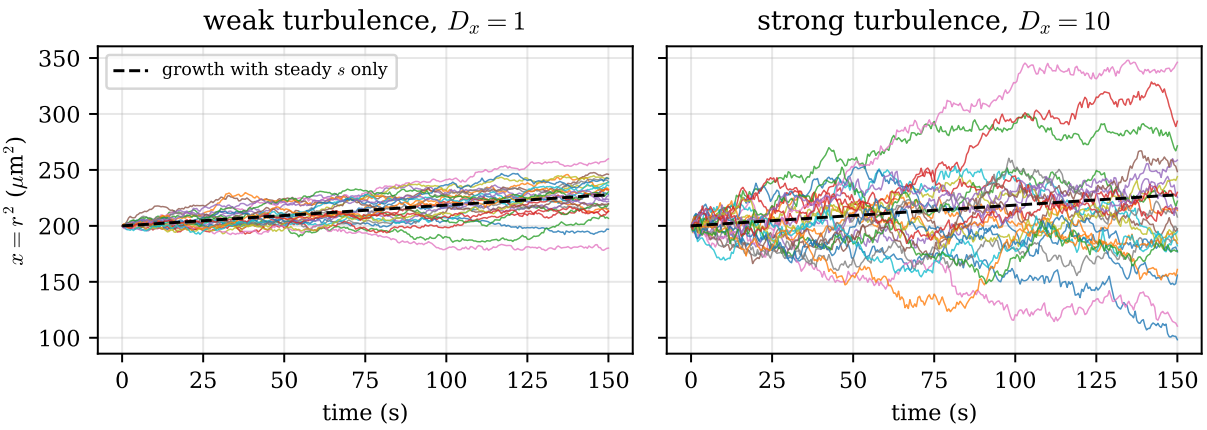


Figure 3: Simulated droplet sizes over time, all starting at $x = 200\,\mu\text{m}^2$. Random fluctuations in growth spread the sizes; larger D_x means faster spreading.

2.4 Falling out

Big droplets fall faster: a droplet's fall speed is proportional to $r^2 = x$. Roughly, a droplet leaves the chamber after the time it takes to fall the chamber's height h. So droplets of size x are removed at the rate

$$\lambda(x) = k\,x \qquad \text{(per second)},$$

with k a known constant (Appendix A). Figure 2(b) shows the typical time $1/\lambda$ a droplet spends in the chamber: minutes for a $10\,\mu$m droplet, much longer for small ones.

2.5 Putting it together

Think of the droplets as traffic on a road that runs from small sizes to large sizes. The **flux** $F(x)$ is the number of droplets (per cm^3 per second) passing size x. Drift carries droplets forward, and diffusion carries them from crowded sizes toward sparse ones:

$$F(x) = \underbrace{A(x)\,N(x)}_{\text{drift}} - \underbrace{D_x \frac{\mathrm{d}N}{\mathrm{d}x}}_{\text{diffusion}}. \tag{2}$$

Now count droplets in a thin slice of sizes $[x, x + \mathrm{d}x]$. At steady state, the number flowing in must equal the number flowing out plus the number falling out of the chamber. That gives the **steady Fokker–Planck equation**

$$\boxed{\frac{\mathrm{d}F}{\mathrm{d}x} + \lambda(x)\,N(x) = 0, \qquad F = A\,N - D_x \frac{\mathrm{d}N}{\mathrm{d}x}.} \tag{3}$$

Two boundary conditions complete it:

- **Injection:** new droplets enter at the smallest size at a fixed rate, $F(x_{\min}) = F_{\text{in}} = 100$ droplets per cm^3 per second.
- **Nothing escapes at the top:** $F(x_{\max}) = 0$, with $x_{\max} = (30\,\mu\text{m})^2$.

The picture is a conveyor belt with a leak. Droplets are loaded at the small end, carried toward larger sizes, jostled back and forth by turbulence, and increasingly likely to fall off as they grow. Integrating (3) over all sizes gives a useful check: everything injected must eventually fall out,

$$F_{\text{in}} = \int \lambda(x)\, N(x)\, \mathrm{d}x. \tag{4}$$

Figure 4 shows steady solutions for several values of D_x. Larger D_x gives a broader distribution that reaches larger sizes. Since large droplets fall out quickly, a broader distribution also holds fewer droplets in total. At very small D_x, droplets pile up near $1\,\mu$m, where the drift is slowest (Figure 2a). These differences are what make D_x recoverable from the distribution.

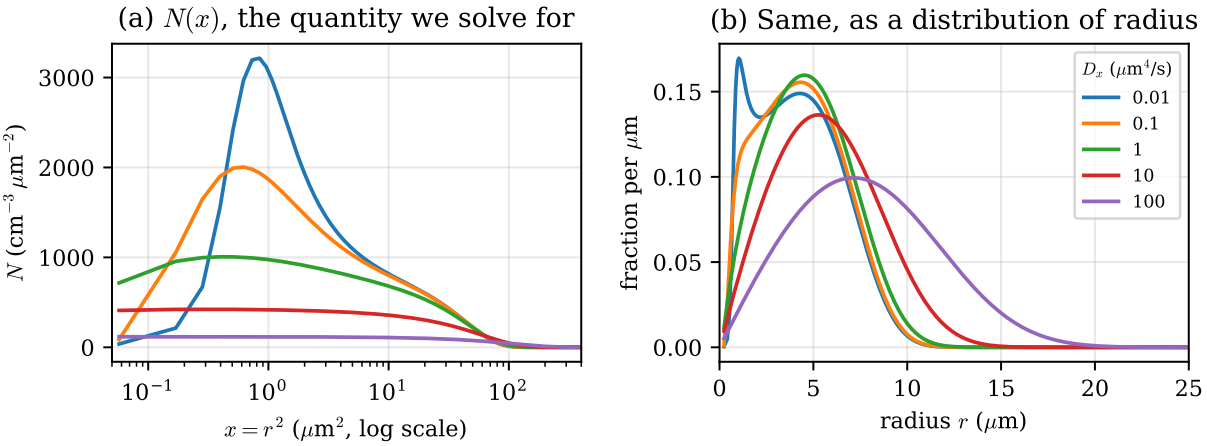


Figure 4: Steady solutions for five values of D_x. (a) $N(x)$ itself. (b) The same solutions as distributions of radius, $2r\,N(r^2)$, each scaled to area 1 so their shapes can be compared.

3 Solving the equation on a computer

3.1 Cells

We divide $[x_{\min}, x_{\max}]$ into $n = 8000$ cells of equal width Δx (Figure 5). The unknown N_i is the average of $N(x)$ over cell i, so $N_i\,\Delta x$ is the concentration of droplets in that cell. The computer solves for the vector $\mathbf{N} = (N_1, \ldots, N_n)$.

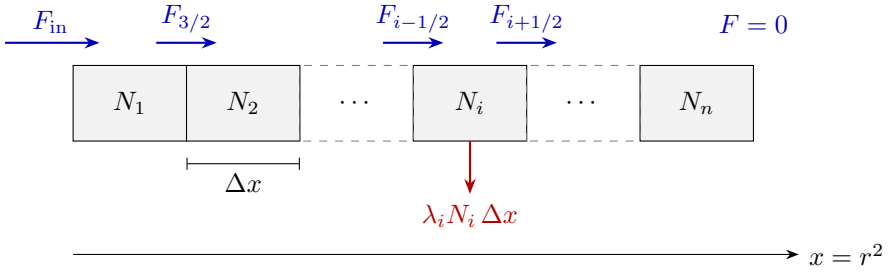


Figure 5: Cells in size space. Droplets enter at the left, flow between neighboring cells (blue), and fall out of every cell (red). Nothing leaves at the right.

3.2 A balance for every cell

Integrating (3) over cell i gives an exact statement: flux out of the right edge, minus flux in through the left edge, plus droplets falling out, equals zero,

$$F_{i+1/2} - F_{i-1/2} + \lambda_i N_i \, \Delta x = 0. \tag{5}$$

Here $F_{i+1/2}$ is the flux across the edge between cells i and $i+1$. The only approximation left is how to compute that flux from the cell values.

3.3 The flux between two cells

The flux (2) involves both N and its slope. The obvious choice, averaging N across the edge for the drift term and using the difference $(N_{i+1} - N_i)/\Delta x$ for the slope, fails when drift is much stronger than diffusion: it produces wiggles and even negative concentrations. Instead we use a formula that is exact in a simple special case.

Between the centers of cells i and $i+1$, treat the drift A and the flux F as constant. Then (2) is a first-order linear ODE for N, $D_x N' = AN - F$, whose solution is

$$N(x) = \frac{F}{A} + C\, e^{A(x - x_i)/D_x}.$$

Requiring $N = N_i$ at x_i and $N = N_{i+1}$ at $x_{i+1} = x_i + \Delta x$, then eliminating C and solving for F, gives the **Scharfetter–Gummel flux**

$$F_{i+1/2} = \alpha_i N_i + \beta_i N_{i+1}, \qquad \alpha_i = \frac{D_x}{\Delta x}\, B(-\mathrm{Pe}_i), \qquad \beta_i = -\frac{D_x}{\Delta x}\, B(\mathrm{Pe}_i), \tag{6}$$

where

$$\mathrm{Pe}_i = \frac{A_{i+1/2}\, \Delta x}{D_x} \quad \text{(the Péclet number)}, \qquad B(z) = \frac{z}{e^z - 1} \quad \text{(the Bernoulli function)}.$$

The Péclet number compares drift to diffusion across one cell. The formula blends two familiar limits:

- **Diffusion dominates** ($|\mathrm{Pe}| \ll 1$): $B \approx 1$, so $F \approx \dfrac{D_x}{\Delta x}(N_i - N_{i+1})$ plus the drift times the average of N_i and N_{i+1}.
- **Drift dominates** ($\mathrm{Pe} \gg 1$): $B(-\mathrm{Pe}) \approx \mathrm{Pe}$ and $B(\mathrm{Pe}) \approx 0$, so $F \approx A\, N_i$. The flux is carried by droplets from the cell they are leaving, which is the physically sensible "upstream" value.

The formula never produces negative concentrations, whatever the value of Pe.

3.4 The linear system

Substituting (6) into (5) gives one linear equation per cell. For an interior cell,

$$-\alpha_{i-1} N_{i-1} + \big(\alpha_i - \beta_{i-1} + \lambda_i \Delta x\big) N_i + \beta_i N_{i+1} = 0.$$

In the first cell the left flux is the injection F_{in}, which moves to the right-hand side. In the last cell the right flux is zero. Altogether,

$$\mathsf{A}(D_x)\, \mathbf{N} = \mathbf{b}, \qquad \mathbf{b} = (F_{\mathrm{in}}, 0, \ldots, 0)^{\top}. \tag{7}$$

Each row involves only a cell and its two neighbors, so A is *tridiagonal* and the system is solved directly in milliseconds. Note two facts used later: D_x enters only through α_i and β_i, and $\mathbf{b}$ does not depend on D_x.

As checks, the computed solutions satisfy the balance (4) to about 12 digits, and give mean radii of $4.22\,\mu$m for $D_x = 0.01$ and $5.74\,\mu$m for $D_x = 10$.

3.5 How fine the grid must be

When drift dominates, the scheme uses the upstream value, and that choice itself smears the distribution slightly, as if there were an extra diffusion of about $|A|\,\Delta x/2$. If the true D_x is much smaller than this, the computed solution barely depends on D_x, and no method can recover it. With 8000 cells ($\Delta x \approx 0.11\,\mu\text{m}^2$), this "numerical" diffusion is about 0.002–0.01 $\mu\text{m}^4/\text{s}$ over most of the size range. It is larger only at the very smallest sizes, where the drift is fast. That's small enough to distinguish every D_x used here, but it sets a floor on the smallest D_x this grid can resolve (Section 7).

4 Setting up the estimation problem

Data. Choose a true value D_x^{true}, solve (7) once, and call the result $\mathbf{N}^{\text{obs}}$. The data are the model's own output in every cell, with no noise. Because the data and the fit use the same model and grid, a correct method must recover D_x^{true} essentially exactly.

Parameter. Instead of D_x we estimate its logarithm,

$$\ell = \ln D_x \qquad (D_x \text{ in } \mu\text{m}^4/\text{s}), \qquad D_x = e^{\ell}.$$

Two reasons: every real ℓ gives a positive D_x, so the optimizer can't step into negative values, and the plausible values of D_x span several powers of ten. In ℓ, doubling D_x is the same step at any scale.

Loss. The loss measures how far the model is from the data:

$$\boxed{\mathcal{J}(\ell) = \frac{1}{2}\,\frac{\sum_i \big(N_i(\ell) - N_i^{\text{obs}}\big)^2}{\sum_i \big(N_i^{\text{obs}}\big)^2}} \tag{8}$$

It is half the squared *relative* mismatch. It's zero only when the model matches the data exactly, and $\mathcal{J} = 0.005$ means about a 10% mismatch. Dividing by $\sum_i (N_i^{\text{obs}})^2$ doesn't move the minimum; it only puts the loss on a readable scale. Its derivative with respect to each model value is

$$\frac{\partial \mathcal{J}}{\partial N_i} = \frac{N_i - N_i^{\text{obs}}}{\sum_k (N_k^{\text{obs}})^2}. \tag{9}$$

Figure 6(a) shows $\mathcal{J}$ as a function of D_x for $D_x^{\text{true}} = 10$. There is a single sharp minimum at the true value, where $\mathcal{J} \approx 5 \times 10^{-25}$, zero to rounding error.

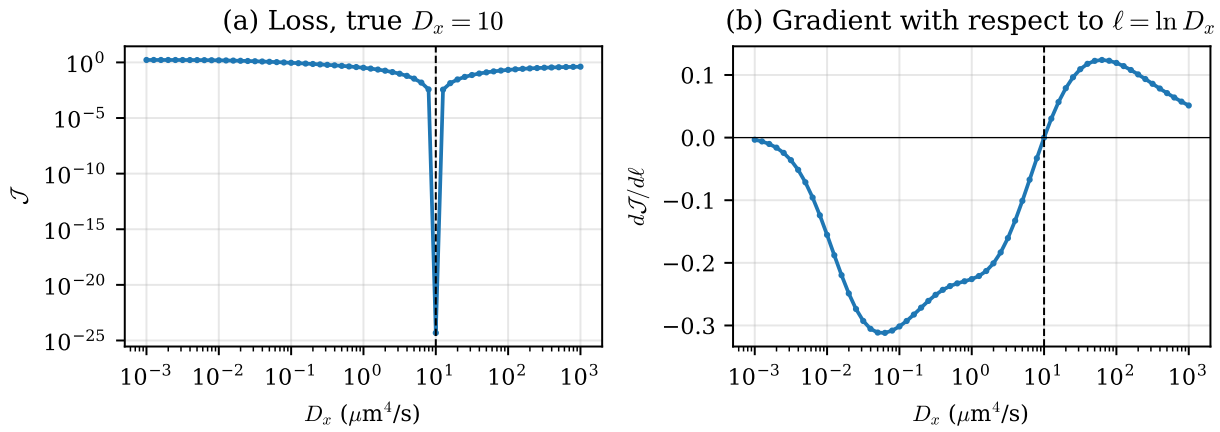


Figure 6: (a) Loss versus D_x when the data were made with $D_x = 10$. (b) The slope of the loss with respect to $\ell = \ln D_x$, computed with the adjoint method. It is negative to the left of the truth and positive to the right, so "downhill" always points toward the answer.

5 The gradient, by the adjoint method

To walk downhill we need the slope $d\mathcal{J}/d\ell$. We could estimate it by solving the model at two nearby values of ℓ (a finite difference). But that gives only an approximation, and it needs extra solves for every unknown parameter. The adjoint method gives the exact slope of the computed loss with one extra solve, no matter how many parameters there are.

5.1 Differentiating the model

The loss depends on D_x only through $\mathbf{N}$. By the chain rule,

$$\frac{d\mathcal{J}}{dD_x} = \mathbf{g}^\top \frac{d\mathbf{N}}{dD_x}, \qquad \mathbf{g} = \frac{\partial \mathcal{J}}{\partial \mathbf{N}} \quad \text{from (9)}.$$

To find $d\mathbf{N}/dD_x$, differentiate $\mathsf{A}\mathbf{N} = \mathbf{b}$. Since $\mathbf{b}$ doesn't depend on D_x,

$$\frac{d\mathsf{A}}{dD_x}\mathbf{N} + \mathsf{A}\frac{d\mathbf{N}}{dD_x} = \mathbf{0} \quad \Longrightarrow \quad \frac{d\mathbf{N}}{dD_x} = -\mathsf{A}^{-1}\frac{d\mathsf{A}}{dD_x}\mathbf{N}.$$

Therefore

$$\frac{d\mathcal{J}}{dD_x} = -\,\mathbf{g}^\top \mathsf{A}^{-1}\,\frac{d\mathsf{A}}{dD_x}\,\mathbf{N}. \tag{10}$$

5.2 The adjoint trick

In (10), the row vector $-\mathbf{g}^\top\mathsf{A}^{-1}$ doesn't depend on which parameter we differentiate with respect to. Give it a name. Define the **adjoint** vector $\boldsymbol{\psi}$ by

$$\boxed{\mathsf{A}^\top\boldsymbol{\psi} = -\,\mathbf{g}} \tag{11}$$

Transposing gives $\boldsymbol{\psi}^\top\mathsf{A} = -\mathbf{g}^\top$, i.e. $\boldsymbol{\psi}^\top = -\mathbf{g}^\top\mathsf{A}^{-1}$. Substituting into (10):

$$\boxed{\frac{d\mathcal{J}}{dD_x} = \boldsymbol{\psi}^\top\frac{d\mathsf{A}}{dD_x}\mathbf{N}, \qquad \frac{d\mathcal{J}}{d\ell} = D_x\frac{d\mathcal{J}}{dD_x}} \tag{12}$$

The second formula is the chain rule for $D_x = e^\ell$. So the gradient costs one solve with the transposed matrix $\mathsf{A}^\top$. The factorization of A computed for the forward solve can be reused for it, so the extra cost is small.

Loss and gradient at a guess ℓ

1. $D_x = e^\ell$. Build $\mathsf{A}(D_x)$ and solve $\mathsf{A}\mathbf{N} = \mathbf{b}$ (forward solve).
2. Compute the loss $\mathcal{J}$ from (8) and $\mathbf{g} = \partial\mathcal{J}/\partial\mathbf{N}$ from (9).
3. Solve $\mathsf{A}^\top\boldsymbol{\psi} = -\mathbf{g}$ (adjoint solve, reusing the factorization of A).
4. Build $d\mathsf{A}/dD_x$ (next subsection) and return $d\mathcal{J}/d\ell = D_x\,\boldsymbol{\psi}^\top(d\mathsf{A}/dD_x)\,\mathbf{N}$.

5.3 The derivative of the matrix

Only α_i and β_i depend on D_x, both directly and through $\mathrm{Pe}_i = A_{i+1/2}\Delta x/D_x$, for which $d\mathrm{Pe}_i/dD_x = -\mathrm{Pe}_i/D_x$. The product and chain rules give

$$\frac{d\alpha_i}{dD_x} = \frac{1}{\Delta x}\Big[B(-\mathrm{Pe}_i) + \mathrm{Pe}_i\,B'(-\mathrm{Pe}_i)\Big], \qquad \frac{d\beta_i}{dD_x} = -\frac{1}{\Delta x}\Big[B(\mathrm{Pe}_i) - \mathrm{Pe}_i\,B'(\mathrm{Pe}_i)\Big], \tag{13}$$

with $B'(z) = \dfrac{1}{e^z-1} - \dfrac{z\,e^z}{(e^z-1)^2}$. The matrix $d\mathsf{A}/dD_x$ has the same tridiagonal pattern as A, with α_i, β_i replaced by (13) and the settling terms replaced by zero (settling doesn't depend on D_x).

5.4 Checking the gradient

Table 1 compares the adjoint gradient with a centered finite difference, $[\mathcal{J}(\ell+\epsilon) - \mathcal{J}(\ell-\epsilon)]/(2\epsilon)$ with $\epsilon = 10^{-5}$. They agree to 7–9 digits; the remaining difference is the finite difference's own error. Figure 6(b) plots the adjoint gradient over the whole range.

D_x	loss $\mathcal{J}$	adjoint $d\mathcal{J}/d\ell$	finite difference	relative difference
0.01	1.539	−0.155349	−0.155349	3.5×10^{-9}
1	0.329	−0.225946	−0.225946	1.3×10^{-8}
5	0.0349	−0.101356	−0.101356	1.3×10^{-7}
30	0.0677	0.106674	0.106674	2.3×10^{-7}
300	0.327	0.087641	0.087641	7.1×10^{-7}

Table 1: Adjoint gradient versus finite differences, with data from $D_x^{\mathrm{true}} = 10$.

5.5 What the adjoint means

Suppose we injected a few extra droplets into cell i, changing $\mathbf{b}$ by $\delta\mathbf{b}$. Then $\delta\mathbf{N} = \mathsf{A}^{-1}\delta\mathbf{b}$, and the loss changes by $\delta\mathcal{J} = \mathbf{g}^\top\mathsf{A}^{-1}\delta\mathbf{b} = -\boldsymbol{\psi}^\top\delta\mathbf{b}$. So ψ_i measures how much adding droplets in cell i would *lower* the loss.

Figure 7 shows this at a wrong guess, $D_x = 20$. The model's distribution is too spread out, so it is short of droplets at small and middle sizes. There $\psi > 0$: adding droplets would improve the fit. The gradient formula (12) then asks how changing D_x would move droplets around, and weights that change by ψ.

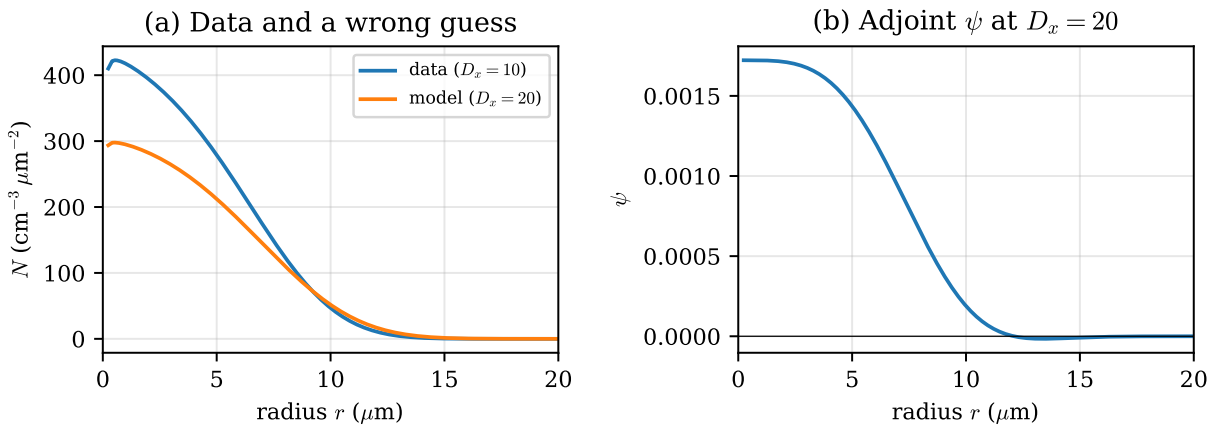


Figure 7: (a) Data ($D_x = 10$) and the model at a wrong guess ($D_x = 20$), plotted against radius. (b) The adjoint at that guess: positive where the model has too few droplets.

6 Minimizing the loss

We use **gradient descent**: repeatedly step downhill,

$$\ell_{\text{new}} = \ell - \alpha\, g, \qquad g = \frac{d\mathcal{J}}{d\ell},$$

where $\alpha > 0$ is the step size. The only subtlety is choosing α: too small wastes iterations, too large overshoots. Three simple rules handle it.

Gradient descent with a line search

Start from a guess ℓ_0, with $\alpha = 1$. Repeat until $|g| < 10^{-10}$:

1. **Propose** the move $\Delta\ell = -\alpha\, g$.
2. **Cap** it: if $|\Delta\ell| > 1$, shrink it to ± 1. So D_x changes by at most a factor of $e \approx 2.7$ per iteration.
3. **Backtrack**: if the loss did not decrease enough, $\mathcal{J}(\ell + \Delta\ell) > \mathcal{J}(\ell) + 10^{-4}\, g\, \Delta\ell$, halve α and go back to step 1.
4. **Accept** the move, and estimate the next step size from how the slope changed: $\alpha = \Delta\ell / \Delta g$, where Δg is the change in gradient. This is the step that would land exactly on the minimum if the loss were a parabola. If $\Delta\ell/\Delta g$ is not positive, double α instead.

The cap prevents wild jumps where the loss is nearly flat. The backtracking guarantees the loss never goes up. The step estimate in rule 4 makes convergence fast near the minimum. Figure 8 shows a run from $D_x = 0.1$ when the truth is 10. For the first few iterations the estimate grows by the maximum factor of e per step. Then the step-size estimate takes over and the estimate closes in within a few more iterations; the loss falls to 3×10^{-25} after 10 iterations.

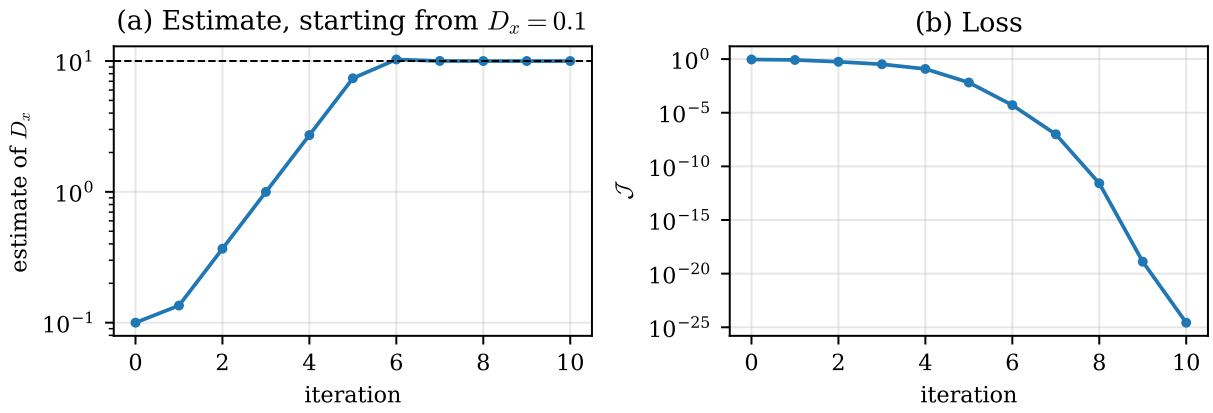


Figure 8: Optimization from $D_x = 0.1$ with true $D_x = 10$. (a) The estimate at each iteration. (b) The loss.

7 Results: many true values, many starting guesses

We repeated the experiment for five true values, $D_x^{\text{true}} \in \{0.01, 0.1, 1, 10, 100\}\,\mu\text{m}^4/\text{s}$, each from seven starting guesses spanning 10^{-3} to 10^{3}. Every one of the 35 runs recovered the true value. The largest relative error was 5×10^{-9}, and no run needed more than 21 iterations (Table 2, Figure 9). Starting guesses farther from the truth take longer, mostly because of the cap of one unit of ℓ per iteration.

	starting guess ($\mu\text{m}^4/\text{s}$)						
true D_x	10^{-3}	10^{-2}	10^{-1}	1	10	10^{2}	10^{3}
0.01	19	0	8	11	13	18	21
0.1	18	11	0	7	10	14	18
1	19	12	8	0	8	12	16
10	20	13	10	7	0	10	13
100	21	16	13	11	9	0	11

Table 2: Iterations needed to converge. Zeros are runs that started at the true value.

A limit: starting too small. Starting guesses of $10^{-4}\,\mu\text{m}^4/\text{s}$ or less fail on this grid. There, the true diffusion is far below the grid's own numerical diffusion (Section 3), so the computed distribution doesn't change when D_x changes. The gradient is essentially zero (3×10^{-15} at $D_x = 10^{-4}$), even though the loss is large (1.66). The optimizer has no downhill direction and stays put. Two remedies are to look at the loss on a coarse grid of D_x values first and start near its minimum, or to use a finer grid if very small D_x matters.

8 Summary

- The size distribution in a steady chamber obeys a drift–diffusion–removal balance, (3). D_x controls how broad it is.

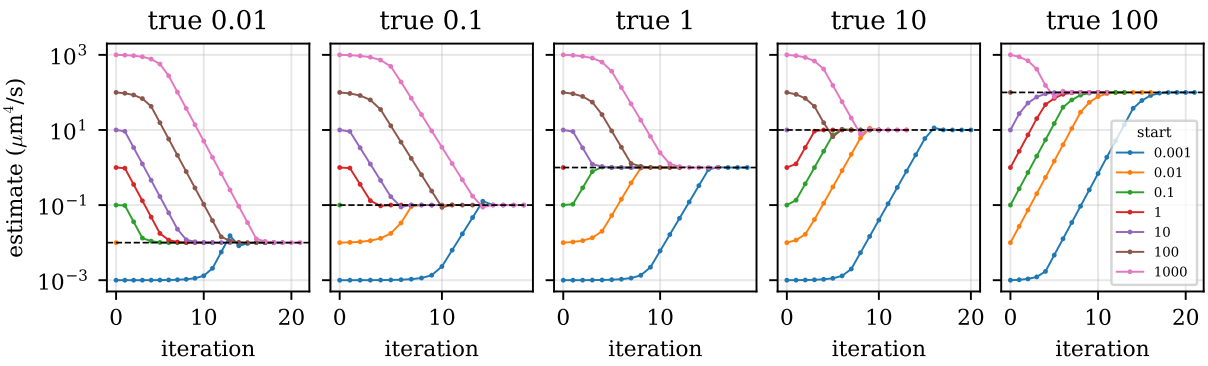


Figure 9: The estimate versus iteration for every run. Each panel is one true value (dashed line); each color is one starting guess.

- Finite volumes with the Scharfetter–Gummel flux turn the equation into a tridiagonal linear system $\mathsf{A}(D_x)\mathbf{N} = \mathbf{b}$.
- The loss is the relative squared mismatch between model and data. Its exact gradient costs one extra solve with $\mathsf{A}^\top$ (the adjoint).
- Gradient descent with a capped, backtracking step recovers D_x to about nine digits from any reasonable starting guess.

What this experiment deliberately leaves out is noise, coarse or incomplete observations, time dependence, other unknown parameters, and data from a model different from the one being fitted. Each of those can be added one at a time on top of the same machinery: the forward solve, loss, adjoint solve, and optimizer stay the same, and only the pieces that change need to be modified.

A Parameter values and formulas

Quantity	Value	Formula / note
Temperature, pressure	$T = 288$ K, $p = 1000$ hPa	
Supersaturation	$s = 0.09\%$	
Growth coefficient	$a = 205.8\,\mu\text{m}^2/\text{s}$	$a = 2\xi$, see below
Köhler curvature term	$A_k = 1.128 \times 10^{-3}\,\mu\text{m}$	$2\sigma_w M_w/(\rho_w RT)$
Köhler salt term	$B_k = 3.650 \times 10^{-4}\,\mu\text{m}^3$	$3\nu m_s M_w/(4\pi\rho_w M_s)$, NaCl, dry radius $0.065\,\mu$m
Critical radius, supersaturation	$0.985\,\mu$m, 0.0763%	peak of $s_{\text{eq}}(r)$
Removal constant	$k = 1.210 \times 10^{-4}\ \mu\text{m}^{-2}\text{s}^{-1}$	$k = 2(\rho_w - \rho_a)g/(9\mu_a h)$, $h = 1$ m
Injection rate	$F_{\text{in}} = 100\ \text{cm}^{-3}\text{s}^{-1}$	
Size range	r from 0.0325 to $30\,\mu$m	$x_{\min} = (0.0325)^2$, $x_{\max} = 30^2\,\mu\text{m}^2$
Grid	$n = 8000$ cells, $\Delta x = 0.1125\,\mu\text{m}^2$	

Growth coefficient. ξ in $r\,\mathrm{d}r/\mathrm{d}t = \xi s$ comes from the rates at which water vapor diffuses to the droplet and latent heat is conducted away:

$$\xi = \left[\rho_w \left(\frac{L^2}{R_v K T^2} + \frac{R_v T}{e_s(T)\, D_v}\right)\right]^{-1},$$

with latent heat $L = 2.5 \times 10^6$ J/kg, water-vapor gas constant $R_v = 461.5$ J/(kg K), thermal conductivity K and vapor diffusivity D_v of air (linear fits in T, with $D_v \propto 1/p$), and saturation vapor pressure $e_s(T)$. The notebook contains the exact expressions.

Symbols in the Köhler and removal formulas. $\sigma_w = 0.075$ N/m (surface tension), $M_w = 18$ g/mol and $M_s = 58.5$ g/mol (molar masses of water and salt), $\rho_w = 1000$ kg/m^3, $R = 8.314$ J/(mol K), $\nu = 2$ (ions per NaCl), $m_s =$ mass of the dry salt particle (density 2160 kg/m^3), $g = 9.81$ m/s^2, air viscosity $\mu_a = 1.8 \times 10^{-5}$ Pa s, air density $\rho_a = p/(287\,T)$.